

Day 1 — SELECT & Basic SQL Queries

Today you'll learn the foundation of SQL: **how to retrieve data from a database.**

Our examples will use a realistic **RAG application database** containing users, documents, and conversations.

1. Today's Goal

By the end of Day 1, you should be able to:

- Understand tables, rows, and columns
- Write SELECT queries
- Select specific columns
- Use *
- Rename columns with AS
- Use expressions in SELECT
- Understand NULL
- Retrieve unique records conceptually
- Debug basic SQL errors
- Read SQL written by another engineer

Important: I will not give you exercise answers until you attempt them.

2. SQL Mental Model

Imagine our RAG application stores documents like this:

documents

id	user_id	title	file_type	page_count
1	101	RAG Architecture	pdf	15
2	101	PostgreSQL Guide	pdf	42
3	102	Python Basics	pdf	28
4	103	FastAPI Notes	md	12

Think of:

- **Database → entire application data**
- **Table → one type of data**
- **Row → one record**
- **Column → one attribute**

For example:

documents

↓

row 3

↓

102 | Python Basics | pdf | 28

3. Your First SQL Query

The basic structure is:

```
SELECT column_name  
FROM table_name;
```

Example:

```
SELECT title  
FROM documents;
```

Result:

RAG Architecture

PostgreSQL Guide

Python Basics

FastAPI Notes

SQL is essentially asking the database:

"Which data do I want?"

4. Selecting Multiple Columns

You can request several columns:

```
SELECT id, title, file_type  
FROM documents;
```

Result:

id	title	file_type
1	RAG Architecture	pdf
2	PostgreSQL Guide	pdf
3	Python Basics	pdf
4	FastAPI Notes	md

The order matters.

```
SELECT title, id  
FROM documents;
```

produces:

title	id
RAG Architecture	1
PostgreSQL Guide	2
Python Basics	3

title	id
FastAPI Notes	4

5. Selecting Everything — *

You can select every column:

```
SELECT *
FROM documents;
```

* means:

all columns

This is useful when exploring a database.

But in production code, avoid blindly using:

```
SELECT *
```

Why?

Imagine your table eventually has:

```
id
user_id
title
file_type
page_count
embedding
metadata
created_at
updated_at
processing_status
...
```

You may only need:

```
SELECT id, title, processing_status
FROM documents;
```

This makes your query's intention clearer and can avoid unnecessarily retrieving large columns such as embeddings or metadata.

6. AS — Renaming Columns

Suppose the application wants the output column to be called `document_title`.

```
SELECT title AS document_title
FROM documents;
```

Result:

document_title

RAG Architecture

PostgreSQL Guide

Python Basics

FastAPI Notes

You can rename multiple columns:

SELECT

id AS document_id,

title AS document_title,

file_type AS document_format

FROM documents;

Important distinction

AS does **not** rename the database column.

It only changes the name in the query result.

The actual table still has:

title

not:

document_title

7. Expressions in SELECT

SQL can perform calculations.

Suppose:

page_count

contains the number of pages.

You can calculate an estimated processing cost:

SELECT

title,

page_count,

page_count * 2 AS estimated_chunks

FROM documents;

If:

page_count = 15

then:

estimated_chunks = 30

The calculated value doesn't automatically become a database column.

It's calculated when the query runs.

8. String Expressions

You can also combine values.

Suppose we have:

users

id	first_name	last_name
101	Om	Kshirsagar
102	Rahul	Patil
103	Priya	Sharma

PostgreSQL supports string concatenation using `||`.

SELECT

first_name || ' ' || last_name AS full_name

FROM users;

Result:

full_name

Om Kshirsagar

Rahul Patil

Priya Sharma

This is useful when constructing display values.

9. NULL

This is extremely important in real databases.

NULL does **not** mean:

0

It does not necessarily mean:

empty string

It means:

the value is unknown / missing / not present.

Example:

documents

id	title	processed_at
1	RAG Architecture	2026-08-29
2	PostgreSQL Guide	2026-08-29
3	Python Basics	NULL

Document 3 may not have been processed yet.

Don't think:

NULL = 0

Think:

NULL = unknown/missing value

We'll work much more deeply with NULL later.

10. SQL Formatting

These two queries are equivalent:

```
SELECT id, title FROM documents;
```

and:

```
SELECT
```

```
id,
```

```
title
```

```
FROM documents;
```

For production SQL, prefer readable formatting:

```
SELECT
```

```
id,
```

```
title,
```

```
file_type,
```

```
page_count
```

```
FROM documents;
```

Good SQL formatting becomes extremely important once queries contain joins, CTEs, and window functions.

11. SQL Is Declarative

This is a major concept.

Python usually tells the computer **how** to perform operations.

SQL generally tells the database **what** result you want.

For example:

```
SELECT title
```

```
FROM documents;
```

You're not telling PostgreSQL:

open table

go to row 1

read title

go to row 2

read title

...

You're declaring:

Give me the titles from documents.

PostgreSQL's query planner decides how to execute that request.

We'll study the planner and EXPLAIN on Day 20.

12. Real RAG Example

Imagine your RAG system has:

users

id

name

email

created_at

documents

id

user_id

title

file_type

page_count

created_at

conversations

id

user_id

title

created_at

messages

id

conversation_id

role

content

created_at

document_chunks

id

document_id

chunk_text

chunk_index

embedding

A basic engineering question might be:

"Show me the IDs and titles of all documents."

SQL:

SELECT

id,

title

FROM documents;

Another:

"Show document title and page count."

SELECT

title,

page_count

FROM documents;

This simple ability is the foundation for everything we'll build later.

13. SQL Comments

Single-line comment:

-- Get document titles

SELECT title

FROM documents;

Multi-line:

/*

Get document information

for debugging

*/

SELECT

id,

title

FROM documents;

Comments are useful when debugging or explaining complicated SQL.

14. Common Day-1 Mistakes

Mistake 1

```
SELECT documents
FROM title;
```

Wrong.

Think:

```
SELECT → columns
FROM → table
```

Correct:

```
SELECT title
FROM documents;
```

Mistake 2

Forgetting the semicolon:

```
SELECT title
FROM documents
```

Many SQL clients will still execute it, but develop the habit of writing:

```
SELECT title
FROM documents;
```

Mistake 3

Using Python-style syntax

```
SELECT(title)
FROM documents;
```

Unless you're calling a SQL function, this isn't how basic column selection works.

Use:

```
SELECT title
FROM documents;
```

Mistake 4

Wrong table name

```
SELECT title
FROM document;
```

when the actual table is:

documents

PostgreSQL will complain that the relation/table doesn't exist.

15. Your First SQL Debugging Task

Suppose PostgreSQL gives an error for:

```
SELECT document_name
FROM documents;
```

The table exists.

Look at the schema:

documents

id

user_id

title

file_type

page_count

created_at

Your task

1. What is wrong?
2. What should the corrected query be?
3. Why does PostgreSQL reject the original query?

Don't look for an answer from me—write it yourself.

16. Exercise Set A — Basic SELECT

Use this fictional dataset:

users

id	name	email	country
1	Aarav	aarav@example.com	India
2	Emma	emma@example.com	USA
3	Liam	liam@example.com	UK
4	Sofia	sofia@example.com	India

documents

id	user_id	title	file_type	page_count
101	1	RAG Fundamentals	pdf	20
102	1	PostgreSQL Guide	pdf	45
103	2	Python Basics	pdf	30
104	3	FastAPI Tutorial	md	15
105	4	Vector Databases	pdf	50

Write SQL for:

Q1. Return every column from users.

Q2. Return only name from users.

Q3. Return id, name, and country from users.

Q4. Return title and page_count from documents.

Q5. Return id, title, and file_type from documents.

Q6. Return all columns from documents.

17. Exercise Set B — Aliases

Write SQL for:

Q7. Return id as user_id and name as user_name.

Q8. Return:

document_id

document_title

pages

using:

id

title

page_count

from documents.

18. Exercise Set C — Expressions

Q9.

Return:

title

page_count

estimated_chunks

Assume:

$\text{estimated_chunks} = \text{page_count} * 2$

Q10.

Return:

name

email

but rename them:

user_name

contact_email

19. Practical Engineering Problem

You're building an admin dashboard for a RAG application.

The frontend needs to display:

Document ID

Document Title

File Type

Pages

Estimated Chunks

The database has:

id

title

file_type

page_count

Task

Write **one SQL query** that produces exactly those five output columns.

Do not use SELECT *.

20. Debugging Challenge

A developer wrote:

```
SELECT
```

```
id document_id,
```

```
title document_title,
```

```
page_count * 2 estimated_chunks
```

```
FROM documents;
```

There is a problem.

Your tasks:

1. Identify the problem.

2. Rewrite the query correctly.
 3. Explain why the original query is problematic.
-

21. Interview Question

Imagine you're interviewing for a junior Data Engineer / GenAI Engineer role.

Question:

What is the difference between `SELECT *` and explicitly selecting columns?

Give me a technical answer.

Don't give me a textbook definition—answer as if you're speaking to an interviewer.

22. Explanation Challenge

Without looking back:

Explain to me:

What happens conceptually when PostgreSQL receives `SELECT title FROM documents`;

Try to explain it in your own words.

You don't need to know the internal query planner yet.

23. Day 1 Short Test

Don't search for answers. Attempt everything.

Q1

What does this return?

```
SELECT title
FROM documents;
```

Q2

What does `*` mean?

```
SELECT *
FROM documents;
```

Q3

What does `AS` do?

```
SELECT title AS document_name
FROM documents;
```

Q4

Is this changing the actual database column name?

```
SELECT title AS document_name  
FROM documents;
```

Answer Yes/No + explain.

Q5

Write a query that returns:

```
id  
title  
page_count  
from documents.
```

Q6

Write a query that returns:

```
title  
estimated_pages  
where:  
estimated_pages = page_count + 5
```

Q7

What is wrong here?

```
SELECT name  
FROM document;
```

Assume the actual table is:

```
documents
```

Q8

What does NULL represent?
